

(SKILL Content Delivery Concepts)

WEEK- 5:

Task 1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Code:

```
#include <stdio.h>

#include <string.h>

int main() {

    /* 1. Apply Single Quotes */
    char ch = 'A';
    printf("1. Single Quote Character: %c\n", ch);

    /* 2. Preserve Literal Content */
    char literal[] = "$name";
    printf("2. Literal Content: %s\n", literal);

    /* 3. Ignore Variable Expansion */
    char name[] = "Rohan";
    char text[] = "$name";
    printf("3. Variable Expansion Ignored: %s\n", text);
    printf("  Actual Variable: %s\n", name);

    /* 4. Store Quoted Strings */
    char message[] = "Hello World";
    printf("4. Stored Quoted String: %s\n", message);

    /* 5. Validate Parsing Results */
    char word[20];
    int result = sscanf("Hello", "%19s", word);
```

```

if (result == 1)
    printf("5. Parsing Result: Success - %s\n", word);
else
    printf("5. Parsing Result: Failed\n");

/* 6. Test Edge Cases */
char empty[] = "";
char special[] = "$ @ # !";

printf("6. Edge Case - Empty String: '%s'\n", empty);
printf("  Edge Case - Special Characters: %s\n", special);

return 0;
}

```

Output:

```

adarohan18@Rohith:~$ nano SingleQuotes.c
adarohan18@Rohith:~$ gcc SingleQuotes.c -o SingleQuotes
adarohan18@Rohith:~$ ./SingleQuotes
1. Single Quote Character: A
2. Literal Content: $name
3. Variable Expansion Ignored: $name
   Actual Variable: Rohan
4. Stored Quoted String: Hello World
5. Parsing Result: Success - Hello
6. Edge Case - Empty String: ''
   Edge Case - Special Characters: $ @ # !
adarohan18@Rohith:~$ |

```

Task 2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Code:

```
#include <stdio.h>

#include <string.h>


int main() {

    /* 1. Apply Double Quotes */

    char message[] = "Hello World";

    printf("1. Double Quoted String: \"%s\"\n", message);


    /* 2. Preserve Spaces */

    char name[] = "Ada Rohan";

    printf("2. Preserved Spaces: \"%s\"\n", name);


    /* 3. Allow Variable Expansion - simulated in C */

    int age = 20;

    printf("3. Variable Expansion: \"Age = %d\"\n", age);


    /* 4. Parse Nested Tokens */

    char input[] = "Hello World";

    char word1[20], word2[20];


    sscanf(input, "%19s %19s", word1, word2);


    printf("4. Nested Tokens: \"%s\" and \"%s\"\n", word1, word2);


    /* 5. Validate Outputs */

    if (strlen(message) > 0)

        printf("5. Output Validation: Success\n");

    else

        printf("5. Output Validation: Failed\n");

}
```

```
/* 6. Test Quoted Commands */  
  
char command[] = "echo \"Hello World\"";  
  
printf("6. Quoted Command: %s\n", command);  
  
return 0;  
}
```

Output:

```
adarohan18@Rohith:~$ nano DoubleQuotes.c  
adarohan18@Rohith:~$ gcc DoubleQuotes.c -o DoubleQuotes  
adarohan18@Rohith:~$ ./DoubleQuotes  
1. Double Quoted String: "Hello World"  
2. Preserved Spaces: "Ada Rohan"  
3. Variable Expansion: "Age = 20"  
4. Nested Tokens: "Hello" and "World"  
5. Output Validation: Success  
6. Quoted Command: echo "Hello World"  
adarohan18@Rohith:~$ |
```

WEEK-6:

Task 1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Code:

```
#include <stdio.h>

int main()
{
    printf("Process Escape Sequences\n");

    printf("Hello\\World\n");
    printf("Hello\\ World\n");
    printf("Special symbols: \\$ \\& \\* \\#\\n");
    printf("Escaped quotes: \\\"Hello\\\"\\n");
    printf("Escaped single quote: \\'Hello\\'\\n");

    printf("\\nComplex Input:\\n");
    printf("This is a\\ test\\ with\\ escaped\\ spaces\\n");

    return 0;
}
```

Output:

```
adarohan18@Rohith:~$ nano escape.c
adarohan18@Rohith:~$ gcc escape.c -o escape
adarohan18@Rohith:~$ ./escape
Process Escape Sequences
Hello\World
Hello\ World
Special symbols: \$ \& \* \#
Escaped quotes: \"Hello\"
Escaped single quote: \'Hello\'

Complex Input:
This is a\ test\ with\ escaped\ spaces
adarohan18@Rohith:~$ |
```

Task 2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Code:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        printf("Process creation failed.\n");
```

```
        return 1;
```

```
    }
```

```
if (pid == 0)
{
    printf("Child Process Created.\n");
    printf("Executing ls command...\n");

    execlp("ls", "ls", "-l", NULL);

    printf("Execution failed.\n");
    return 1;
}
else
{
    printf("Parent Process.\n");
    printf("Child PID: %d\n", pid);

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
}
```

Output:

```
adaroohan18@Rohith:~$ nano process.c
adaroohan18@Rohith:~$ gcc process.c -o process
adaroohan18@Rohith:~$ ./process
Parent Process.
Child Process Created.
Executing ls command...
Child PID: 593
total 700
-rwxr-xr-x 1 adaroohan18 adaroohan18 16520 Aug 25 03:57 CommandExecutor
-rw-r--r-- 1 adaroohan18 adaroohan18 1235 Aug 25 03:54 CommandExecutor.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16096 Sep 15 02:53 FCFS
-rw-r--r-- 1 adaroohan18 adaroohan18 676 Sep 15 02:53 FCFS.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16320 Aug 25 04:06 FileCopySysCalls
-rw-r--r-- 1 adaroohan18 adaroohan18 1469 Aug 25 04:05 FileCopySysCalls.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16216 Sep 20 14:43 MemoryAlloc
-rw-r--r-- 1 adaroohan18 adaroohan18 1155 Sep 20 14:42 MemoryAlloc.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16256 Sep 20 14:35 MemorySegments
-rw-r--r-- 1 adaroohan18 adaroohan18 693 Sep 20 14:34 MemorySegments.c
prw-r--r-- 1 adaroohan18 adaroohan18 0 Aug 28 04:03 NamedPipe.fifo
drwxr-xr-x 4 adaroohan18 adaroohan18 4096 Aug 4 03:51 OSSP
-rwxr-xr-x 1 adaroohan18 adaroohan18 16096 Sep 15 03:59 Priority
-rw-r--r-- 1 adaroohan18 adaroohan18 1606 Sep 15 03:59 Priority.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16104 Sep 15 04:06 RoundRobin
-rw-r--r-- 1 adaroohan18 adaroohan18 1769 Sep 15 04:05 RoundRobin.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16096 Sep 15 03:02 SJF_NP
-rw-r--r-- 1 adaroohan18 adaroohan18 927 Sep 15 03:01 SJF_NP.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16128 Aug 25 03:18 SystemCallClose
-rw-r--r-- 1 adaroohan18 adaroohan18 493 Aug 25 03:17 SystemCallClose.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16128 Aug 18 03:30 SystemCallOpen
-rw-r--r-- 1 adaroohan18 adaroohan18 399 Aug 18 03:28 SystemCallOpen.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16184 Aug 18 03:43 SystemCallRead
-rw-r--r-- 1 adaroohan18 adaroohan18 641 Aug 18 03:42 SystemCallRead.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16184 Aug 18 03:53 SystemCallWrite
-rw-r--r-- 1 adaroohan18 adaroohan18 540 Aug 18 03:52 SystemCallWrite.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16312 Aug 25 02:54 SystemCalls
-rw-r--r-- 1 adaroohan18 adaroohan18 1481 Aug 25 02:54 SystemCalls.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 16312 Aug 25 02:52 a.out
-rwxr-xr-x 1 adaroohan18 adaroohan18 16384 Aug 28 09:09 client
-rw-r--r-- 1 adaroohan18 adaroohan18 945 Aug 28 09:08 client.c
-rwxr-xr-x 1 adaroohan18 adaroohan18 15952 Sep 20 16:10 escape
-rw-r--r-- 1 adaroohan18 adaroohan18 397 Sep 20 16:09 escape.c
```

```
-rwxr-xr-x 1 adarohan18 adarohan18 16176 Sep 20 14:55 forkProcMemo
-rw-r--r-- 1 adarohan18 adarohan18 1063 Sep 20 14:54 forkProcMemo.c
-rw-r--r-- 1 adarohan18 adarohan18 16184 Aug 25 02:50 hello.c
-rw-r--r-- 1 adarohan18 adarohan18 138 Aug 28 03:32 info.txt
-rw-r--r-- 1 adarohan18 adarohan18 117 Aug 18 03:46 input.txt
-rw----- 1 adarohan18 adarohan18 1 Aug 28 03:30 osp.c.save
-rw-r--r-- 1 adarohan18 adarohan18 56 Aug 18 03:55 output.txt
-rwxr-xr-x 1 adarohan18 adarohan18 16000 Aug 3 04:56 practical1
-rw-r--r-- 1 adarohan18 adarohan18 103 Aug 3 04:55 practical1.c
-rw-r--r-- 1 adarohan18 adarohan18 103 Aug 25 02:57 practical2.c
-rwxr-xr-x 1 adarohan18 adarohan18 16120 Sep 20 16:25 process
-rw-r--r-- 1 adarohan18 adarohan18 642 Sep 20 16:23 process.c
-rw-r--r-- 1 adarohan18 adarohan18 641 Sep 20 16:17 process.c.save
-rw----- 1 adarohan18 adarohan18 605 Sep 20 16:19 program.c.save
-rwxr-xr-x 1 adarohan18 adarohan18 16352 Aug 11 03:37 program10
-rw-r--r-- 1 adarohan18 adarohan18 1064 Aug 11 03:35 program10.c
-rwxr-xr-x 1 adarohan18 adarohan18 16208 Aug 11 03:53 program11
-rw-r--r-- 1 adarohan18 adarohan18 977 Aug 11 03:53 program11.c
-rwxr-xr-x 1 adarohan18 adarohan18 16000 Aug 3 05:20 program2
-rw-r--r-- 1 adarohan18 adarohan18 98 Aug 3 05:19 program2.c
-rwxr-xr-x 1 adarohan18 adarohan18 16000 Aug 3 05:31 program3
-rw-r--r-- 1 adarohan18 adarohan18 175 Aug 3 05:30 program3.c
-rwxr-xr-x 1 adarohan18 adarohan18 16088 Aug 3 05:40 program4
-rw-r--r-- 1 adarohan18 adarohan18 75 Aug 18 03:13 program4.c
-rwxr-xr-x 1 adarohan18 adarohan18 16088 Aug 11 03:38 program5
-rw-r--r-- 1 adarohan18 adarohan18 239 Aug 3 05:47 program5.c
-rwxr-xr-x 1 adarohan18 adarohan18 16040 Aug 3 05:52 program6
-rw-r--r-- 1 adarohan18 adarohan18 122 Aug 3 05:51 program6.c
-rwxr-xr-x 1 adarohan18 adarohan18 16040 Aug 3 06:03 program7
-rw-r--r-- 1 adarohan18 adarohan18 246 Aug 3 06:02 program7.c
-rwxr-xr-x 1 adarohan18 adarohan18 16040 Aug 3 06:09 program8
-rw-r--r-- 1 adarohan18 adarohan18 154 Aug 3 06:08 program8.c
-rwxr-xr-x 1 adarohan18 adarohan18 16352 Aug 25 04:08 program9
-rw-r--r-- 1 adarohan18 adarohan18 1116 Aug 11 03:15 program9.c
-rwxr-xr-x 1 adarohan18 adarohan18 16472 Aug 28 09:06 server
-rw-r--r-- 1 adarohan18 adarohan18 1190 Aug 28 09:06 server.c
-rwxr-xr-x 1 adarohan18 adarohan18 16304 Aug 25 03:40 shell
-rw-r--r-- 1 adarohan18 adarohan18 517 Aug 25 03:39 shell.c
-rwxr-xr-x 1 adarohan18 adarohan18 16288 Aug 28 09:39 signal
-rw-r--r-- 1 adarohan18 adarohan18 1124 Aug 28 09:39 signal.c
-rw----- 1 adarohan18 adarohan18 1525 Aug 28 09:36 signal.c.save
```

```
-rw-r--r-- 1 adarohan18 adarohan18 1124 Aug 28 09:39 signal.c
-rw----- 1 adarohan18 adarohan18 1525 Aug 28 09:36 signal.c.save
drwx----- 3 adarohan18 adarohan18 4096 Aug 4 03:26 snap
-rw-r--r-- 1 adarohan18 adarohan18 0 Aug 18 03:30 test.txt
Child process completed.
adarohan18@Rohith:~$ |
```

WEEK-7:

Task 1: Process Synchronization and Child Process Monitoring Using waitpid().

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        printf("Process creation failed.\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process Started.\n");
        printf("Child PID: %d\n", getpid());

        sleep(2);

        printf("Child Process Completed.\n");
        return 10;
    }
    else
    {
        printf("Parent Process Started.\n");
        printf("Waiting for child process...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child process exited normally.\n");
            printf("Child exit status: %d\n",
                WEXITSTATUS(status));
        }

        printf("Parent Process Completed.\n");
    }
}
```

```
    return 0;
}
```

Output:

```
adarohan18@Rohith:~$ nano wait.c
adarohan18@Rohith:~$ gcc wait.c -o wait
adarohan18@Rohith:~$ ./wait
Parent Process Started.
Waiting for child process...
Child Process Started.
Child PID: 700
Child Process Completed.
Child process exited normally.
Child exit status: 10
Parent Process Completed.
adarohan18@Rohith:~$ |
```

Task 2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

int main()
{
    char *path;
    char *path_copy;
    char *directory;
    char command[100];
    char full_path[500];
    int found = 0;

    path = getenv("PATH");

    if (path == NULL)
    {
```

```
    printf("PATH variable not found.\n");
    return 1;
}

path_copy = strdup(path);

if (path_copy == NULL)
{
    printf("Memory allocation failed.\n");
    return 1;
}

printf("Enter command to search: ");
scanf("%99s", command);

directory = strtok(path_copy, ":");

while (directory != NULL)
{
    snprintf(full_path, sizeof(full_path),
             "%s/%s", directory, command);

    if (access(full_path, X_OK) == 0)
    {
        printf("Executable found: %s\n", full_path);
        found = 1;
        break;
    }

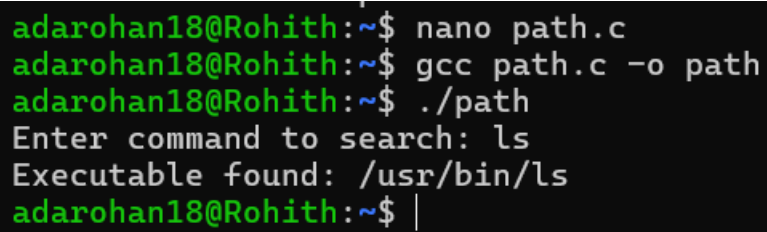
    directory = strtok(NULL, ":");
}
```

```
if (!found)
{
    printf("Command not found: %s\n", command);
}

free(path_copy);

return 0;
}
```

Output:



```
adarohan18@Rohith:~$ nano path.c
adarohan18@Rohith:~$ gcc path.c -o path
adarohan18@Rohith:~$ ./path
Enter command to search: ls
Executable found: /usr/bin/ls
adarohan18@Rohith:~$ |
```
